

4.1 Guía de Funciones: Importancia de Variables y Feature Selection

R — caret + vip + DALEX

```
library(caret)
library(vip)      # variable importance plots
library(randomForest)

data(PimaIndiansDiabetes, package="mlbench")

# 1. IMPORTANCIA POR REGRESIÓN LOGÍSTICA
# -----
m_lr <- train(diabetes ~ ., data=PimaIndiansDiabetes,
              method="glm", family="binomial",
              preProcess=c("center","scale"),
              trControl=trainControl(method="cv", number=5))

# Importancia absoluta de coeficientes
vip(m_lr, num_features=8, geom="col",
     aesthetics=list(fill="#6A1B9A", alpha=0.8)) +
  ggtitle("Importancia de variables – Regresión Logística")

# 2. IMPORTANCIA EN RANDOM FOREST
# -----
m_rf <- train(diabetes ~ ., data=PimaIndiansDiabetes,
              method="rf", ntree=200,
              trControl=trainControl(method="cv", number=5))
vip(m_rf, num_features=8) # Gini importance

# Permutation importance (más confiable)
library(DALEX)
explainer <- explain(m_rf$finalModel,
                     data=PimaIndiansDiabetes[, -9],
```

```

        y=as.numeric(PimaIndiansDiabetes$diabetes=="pos"),
        label="Random Forest")
vi_permutation <- model_parts(explainer, type="difference")
# type="difference": caída en AUC al permutar
plot(vi_permutation)

# 3. FEATURE SELECTION CON RFE
# -----
ctrl_rfe <- rfeControl(functions=rfFuncs,
                      method="cv", number=5,
                      verbose=FALSE)
# rfFuncs: usa Random Forest para evaluar importancia
# También disponible: lmFuncs (LR), nbFuncs (NB)

rfe_result <- rfe(PimaIndiansDiabetes[,-9], # features
                 PimaIndiansDiabetes[,9],  # respuesta
                 sizes=c(2:8),             # probar 2 a 8 features
                 rfeControl=ctrl_rfe)

print(rfe_result)
predictors(rfe_result) # features seleccionadas
plot(rfe_result, type=c("g","o")) # accuracy vs nro features

# 4. FILTER METHOD: Chi-cuadrado
# -----
# Selección univariada por chi-cuadrado
data_disc <- PimaIndiansDiabetes
for (col in names(data_disc)[-9]) {
  data_disc[[col]] <- cut(data_disc[[col]], breaks=4, labels=FALSE)
}
chi_scores <- sapply(names(data_disc)[-9], function(col) {
  ct <- table(data_disc[[col]], data_disc$diabetes)
  chisq.test(ct)$statistic
})
sort(chi_scores, decreasing=TRUE)

```

Python — sklearn + shap

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE, SelectKBest, chi2, mutual_info_classif
from sklearn.preprocessing import StandardScaler

```

```
from sklearn.datasets import load_breast_cancer
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import shap

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names

# Escalar
scaler = StandardScaler()
X_s = scaler.fit_transform(X)

# 1. IMPORTANCIA – REGRESIÓN LOGÍSTICA
lr = LogisticRegression(max_iter=5000, random_state=42)
lr.fit(X_s, y)
imp_df = pd.DataFrame({'feature': feature_names, 'importance': np.abs(lr.coef_[0])})
imp_df = imp_df.sort_values('importance', ascending=False)
imp_df.head(10).plot(kind='barh', x='feature', y='importance',
                      color='#6A1B9A', legend=False, figsize=(8,6))
plt.title('Importancia – Regresión Logística (|coeficiente|)')
plt.tight_layout()

# 2. IMPORTANCIA – RANDOM FOREST (Gini)
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X, y)
rf_imp = pd.Series(rf.feature_importances_, index=feature_names).sort_values(ascending=False)
rf_imp.head(10).plot(kind='bar', color='#9C27B0', figsize=(10,5))
plt.title('Importancia Gini – Random Forest')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()

# 3. SHAP VALUES
explainer = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X[:100]) # primeras 100 muestras

# Summary plot (importancia global)
shap.summary_plot(shap_values[1], X[:100], feature_names=feature_names)

# Waterfall para una prediccion individual
shap.waterfall_plot(shap.Explanation(values=shap_values[1][0],
                                   base_values=explainer.expected_value[1],
                                   data=X[0],
                                   feature_names=feature_names))

# 4. RFE con cross-validation
from sklearn.feature_selection import RFECV
```

```
rfecv = RFECV(estimator=LogisticRegression(max_iter=1000),  
              step=1, cv=5, scoring='accuracy')  
rfecv.fit(X_s, y)  
print(f"Número óptimo de features: {rfecv.n_features_}")  
print("Features seleccionadas:")  
print([f for f, s in zip(feature_names, rfecv.support_) if s])
```